

BUREAU D'ETUDE - RUCHE CONNECTEE

Rapport Technique

Module de Communication LoRa-E5 - Mode Point à Point (P2P)

Binome : AGAMA Justin

Encadrant : PERISSE Thierry



Module :	Grove LoRa-E5-HF (Seeed Studio)
Puce :	STM32WLE5JC (MCU + SX126X intégrés)
Interface :	UART : Commandes AT
Fréquence :	868,3 MHz (bande EU868)
Mode étudié :	P2P (Point à Point : mode TEST)
Application :	Ruche connectée : transmission données capteurs
Date :	Mars 2026

Sommaire

1. Introduction : La ruche connectée et LoRa
2. Présentation matérielle du module Grove LoRa-E5
3. Technologie LoRa et mode P2P
4. Protocole de communication AT
5. Architecture du driver logiciel
6. Description détaillée des fonctions
7. Séquence complète d'une transaction P2P
8. Intégration dans le projet ruche connectée
9. Résultats expérimentaux
10. Perspectives : Mode LoRaWAN

1. Introduction : La ruche connectée et LoRa

Ce rapport présente le module de communication LoRa-E5 utilisé dans le cadre du projet de ruche connectée. L'objectif du système est de collecter en continu les données environnementales de la ruche : température extérieure et intérieure et masse puis de les transmettre sans fil à une unité de réception, qui les affiche sur un écran LCD local et les enregistre.

Le choix de la technologie LoRa (Long Range) est motivé par ses propriétés intrinsèques particulièrement adaptées à ce type d'application :

- Longue portée : plusieurs centaines de mètres en environnement urbain, plusieurs kilomètres en espace ouvert idéal pour des ruches situées en milieu agricole éloigné.
- Faible consommation énergétique : le module peut opérer sur batterie pendant de longues périodes, la consommation en veille descendant à 1,4 μ A.
- Aucune infrastructure réseau requise en mode P2P : deux modules communiquent directement, sans gateway ni connexion internet.
- Robustesse aux perturbations : la modulation par étalement de spectre (Spread Spectrum) confère une excellente immunité au bruit et aux interférences.

Le mode Point à Point (P2P) est celui retenu pour ce projet. Il utilise le mode TEST du firmware LoRa-E5, qui permet une communication directe entre deux modules sans passer par un réseau LoRaWAN.

2. Présentation matérielle du module Grove LoRa-E5

2.1 Vue d'ensemble

Le module utilisé est le Grove LoRa-E5-HF de Seeed Studio. Il s'agit d'un module de développement compact intégrant directement sur un PCB le module LoRa-E5 de ST Microelectronics, une antenne filaire quart d'onde pour la bande haute fréquence (HF, au-dessus de 525 MHz), et un connecteur Grove 4 broches pour l'interface UART.

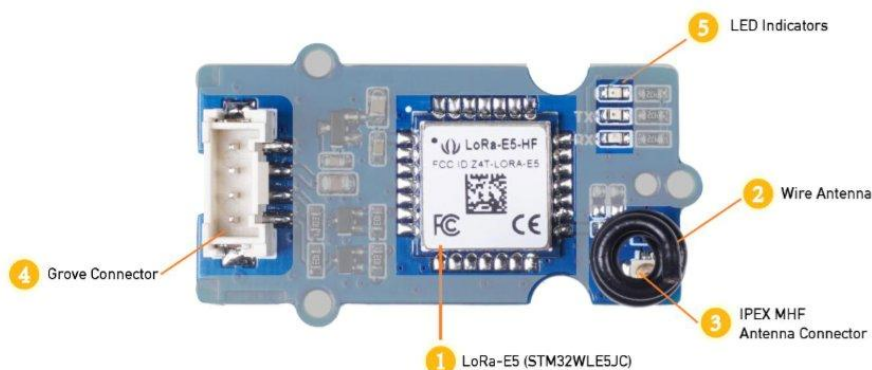


Figure 1 — Module Grove LoRa-E5-HF annoté (source : Seeed Studio)

Le module LoRa-E5 lui-même est une solution System-in-Package (SiP) : il intègre dans un unique boîtier CMS de 12 × 12 mm le microcontrôleur STM32WLE5JC et le transceiver LoRa SX126X de Semtech.

2.2 Caractéristiques techniques

Paramètre	Valeur
Puce principale	STM32WLE5JC (ARM Cortex-M4 + SX126X)
Bande de fréquence (HF)	868 MHz / 915 MHz
Alimentation	3,3 V (compatible 3,3 V MCU)
Puissance d'émission max.	22 dBm (HF et LF)
Sensibilité en réception	-136 dBm @ SF12 BW125
Courant veille (watchdog ON)	1,9 µA @ 3,3 V
Interface MCU hôte	UART (commandes AT), 9600 baud 8N1 par défaut
Mémoire EEPROM utilisateur	256 octets non-volatils
Certifications	CE, FCC
Firmware	Version 4.0.11 (utilisée dans ce projet)

2.3 Schéma de câblage : Connexion P2P

En mode P2P, deux modules Grove LoRa-E5 sont chacun connectés à leur MCU STM32 hôte via l'interface UART. La communication RF s'effectue directement entre les deux modules sur la fréquence configurée, sans aucun équipement intermédiaire.

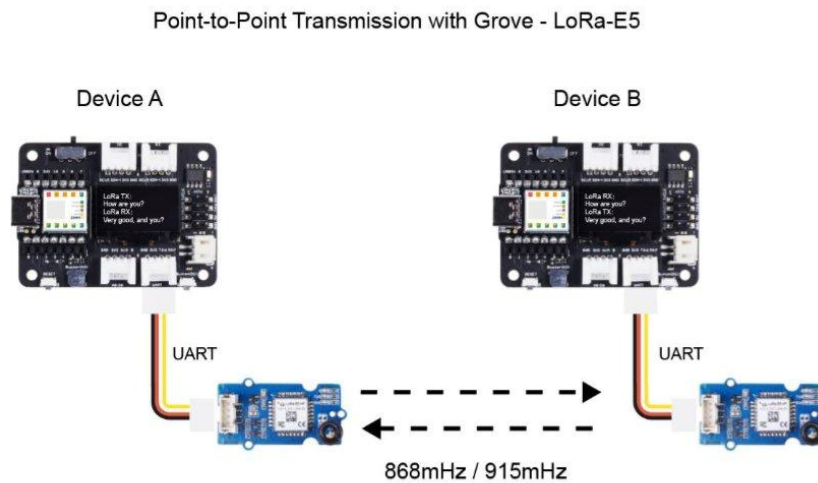


Figure 2 — Architecture Point à Point avec deux modules Grove LoRa-E5 (source : Seeed Studio)

3. Technologie LoRa et mode P2P

3.1 Principe de la modulation LoRa

LoRa (Long Range) est une technique de modulation propriétaire développée par Semtech, basée sur le principe du Chirp Spread Spectrum (CSS). Contrairement aux modulations classiques (FSK, GFSK) qui concentrent l'énergie sur une bande étroite, LoRa étale le signal sur une large bande passante en faisant varier linéairement la fréquence instantanée (chirp). Cette technique confère trois propriétés fondamentales :

- Robustesse : le signal LoRa reste décodable même sous le niveau du bruit ambiant (SNR négatif possible).
- Longue portée : un bilan de liaison très favorable permet de couvrir de grandes distances avec une puissance d'émission modérée.
- Immunité aux interférences : l'étalement spectral rend le signal quasi imperceptible pour les systèmes non-LoRa.

3.2 Paramètres RF : Configuration du projet

Trois paramètres principaux gouvernent les performances d'un lien LoRa. La configuration utilisée dans ce projet est la suivante :

Paramètre	Valeur utilisée	Explication
Fréquence	868,3 MHz	Canal central de la bande EU868 (Europe)
Spreading Factor (SF)	SF12	Portée maximale, débit le plus faible (250 bit/s)
Bandwidth (BW)	125 kHz	Largeur de bande standard LoRaWAN EU868
Preamble TX/RX	8 symboles	Durée de préambule pour la synchronisation
Puissance TX	20 dBm	100 mW, puissance élevée pour maximiser la portée
CRC	ON	Vérification d'intégrité activée sur chaque trame RF
IQ inversé	OFF	Mode standard (non inversé, même config TX et RX)
Réseau public	OFF	Mode privé, pas de syncword LoRaWAN public

Pourquoi SF12 ? Le Spreading Factor 12 offre la portée maximale mais le débit le plus faible (~250 bit/s). Pour la ruche, les trames sont courtes (~50 caractères) et la périodicité d'envoi est de l'ordre de plusieurs secondes. SF12 est donc le choix optimal : on maximise la robustesse du lien et la portée, sans contrainte sur le débit.

3.3 Mode P2P vs LoRaWAN

Critère	Mode P2P (utilisé)	Mode LoRaWAN
Infrastructure	Aucune, 2 modules	Gateway + serveur réseau requis
Portée	Limitée par la puissance RF	Étendue via le réseau de gateways
Sécurité	Aucun chiffrement natif	AES-128 (NwkSKey / AppSKey)
Débit utile	Jusqu'à 255 octets par trame	Limité par le duty cycle réglementaire
Complexité	Faible : commandes AT simples	Élevée : join, sessions, MAC layer
Usage typique	Lien local dédié, prototypage	IoT industriel, déploiements à grande échelle

4. Protocole de communication AT

4.1 Conventions

Le module LoRa-E5 est piloté via des commandes AT (Hayes command set) transmises sur l'interface UART. Les conventions à respecter sont les suivantes :

- Chaque commande doit se terminer par `\r\n` (CR LF).
- Les commandes sont insensibles à la casse (AT = at = At).
- Chaque commande génère systématiquement une réponse du module. (utilisé pour le contrôle de la transmission et de la réception.)
- La longueur maximale d'une commande est de 528 caractères.
- La configuration UART par défaut est : 9600 baud, 8 bits de données, sans parité, 1 bit de stop (8N1).

Le format général des échanges est le suivant :

```
Envoi MCU → AT+COMMANDE=PARAMETRE\r\n
Réponse ← +COMMANDE: VALEUR

Exemple :
Envoi → AT\r\n
Réponse ← +AT: OK
```

4.2 Commandes AT utilisées dans le projet

Commande envoyée	ACK attendu	Rôle dans le projet
AT\r\n	+AT: OK	Test de présence du module
AT+VER\r\n	+VER:	Lecture version firmware
AT+ID\r\n	+ID:	Lecture DevAddr / DevEui / AppEui
AT+MODE=TEST\r\n	+MODE: TEST	Passage en mode P2P (TEST)
AT+TEST=RFCFG,...\r\n	+TEST: RFCFG ...	Configuration radio (fréquence, SF, BW, puissance)
AT+TEST=RXLRPKT\r\n	+TEST: RXLRPKT	Mise en écoute continue
AT+TEST=TXLRSTR,\"...\"...\r\n	+TEST: TXLRSTR	Émission d'une chaîne de caractères
AT+LOWPOWER\r\n	(pas d'ACK attendu)	Mise en veille du module
AT+RESET\r\n		Réinitialisation logicielle
AT+FDEFAULT\r\n	+FDEFAULT: OK	Remise aux paramètres usine

5. Architecture du driver logiciel

5.1 Principes de conception

Le driver LoRa-E5 est développé pour STM32 avec la bibliothèque HAL (Hardware Abstraction Layer). Il repose sur deux structures de données centrales et un ensemble de fonctions publiques et privées organisées.

Le choix d'une architecture orientée handle (plutôt que des variables globales) est délibéré : il permet d'instancier plusieurs modules LoRa indépendants sur un même MCU sans conflit d'état. Cette flexibilité est gérée par la constante NUM_LORA_MODULES et le tableau de pointeurs lora_handles[], utilisés dans le callback d'interruption UART pour router les données vers le bon handle.

5.2 Structure : LORA Handle t

C'est la structure principale qui encapsule tout l'état d'un module LoRa. Elle est passée par pointeur à chaque fonction du driver.

```
typedef struct {
    UART_HandleTypeDef *huart;           // Handle UART associe
    volatile uint8_t rx_byte;             // Octet courant reçu en IT
    volatile uint8_t rx_flag;             // Indicateur de reception
    char rx_buffer[256];                 // Buffer de reception complet
    uint16_t rx_index;                   // Index courant dans le buffer
    char tx_buffer[256];                 // Buffer d'emission
    LORA_State_t state;                 // Etat du module
    (IDLE/RX/TX/SLEEP)
    LORA_Mode_t mode;                   // Mode (P2P / LoRaWAN)
    LORA_P2P_Packet_t last_packet;       // Dernier paquet reçu
} LORA_Handle_t;
```

Le champ rx_buffer est alimenté caractère par caractère par le callback d'interruption UART HAL_UART_RxCpltCallback(). Les champs volatiles rx_byte et rx_flag signalent à la boucle principale qu'un octet est disponible, nécessitant une protection par sections critiques (__disable_irq() / __enable_irq()) lors des accès concurrents pour ne pas perdre le fil de lecture et d'écriture des données dans le buffer.

5.3 Structure : LORA P2P Packet t

Cette structure représente un paquet LoRa reçu et est remplie par la fonction LORA_P2P_Read() après parsing de la réponse AT.

```
typedef struct {
    uint8_t payload[256];               // Donnees recues (converties de l'hex)
    uint16_t length;                   // Longueur effective du payload en octets
    int16_t rssi;                      // Niveau de signal reçu (dBm)
    int8_t snr;                        // Rapport signal/bruit (dB)
} LORA_P2P_Packet_t;
```

5.4 Codes de retour : LORA Status t

Code	Signification
LORA_STATUS_OK	Opération réussie : ACK reçu
LORA_STATUS_ERROR	Erreur générique : réponse invalide
LORA_STATUS_TIMEOUT	Pas de réponse dans le délai imparti
LORA_STATUS_BUSY	Module occupé : transaction en cours
LORA_STATUS_INVALID_PARAM	Paramètre nul ou invalide passé à la fonction

6. Description détaillée des fonctions

6.1 LORA Init() : Initialisation du module

```
LORA_Status_t LORA_Init(LORA_Handle_t *hlora, UART_HandleTypeDef *huart)
{
    if (!hlora || !huart) return LORA_STATUS_INVALID_PARAM;
    memset(hlora, 0, sizeof(LORA_Handle_t));
    hlora->huart = huart;
    hlora->state = LORA_STATE_IDLE;
    hlora->mode = LORA_MODE_P2P;
    hlora->rx_index = 0;
    HAL_UART_Receive_IT(huart, (uint8_t*)&hlora->rx_byte, 1);
    return LORA_STATUS_OK;
}
```

Cette fonction initialise l'intégralité du handle à zéro (memset), puis configure les champs essentiels. L'appel à HAL_UART_Receive_IT() est fondamental : il active la réception UART par interruption en mode 1 octet. À chaque octet reçu, le callback HAL_UART_RxCpltCallback() sera déclenché automatiquement par le hardware, sans bloquer le programme principal. La réception est ainsi permanente et non-bloquante dès l'initialisation.

6.2 HAL_UART_RxCpltCallback() : Réception par interruption

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    extern LORA_Handle_t *lora_handles[];
    for (int i = 0; i < NUM_LORA_MODULES; i++)
    {
        LORA_Handle_t *hlora = lora_handles[i];
        if (hlora && hlora->huart == huart)
        {
            hlora->rx_buffer[hlora->rx_index++] = hlora->rx_byte;
            if (hlora->rx_index >= (LORA_RX_BUFFER_SIZE - 1))
            {
                hlora->rx_index = 0;
            }
            hlora->rx_buffer[hlora->rx_index] = '\0';
            HAL_UART_Receive_IT(hlora->huart, (uint8_t*)&hlora->rx_byte, 1);
            break;
        }
    }
}
```

Ce callback, déclenché à chaque octet reçu sur l'UART, accumule les caractères dans rx_buffer en maintenant un index courant. Le caractère null terminal est placé après chaque octet pour que le buffer reste toujours une chaîne valide. La boucle sur lora_handles[] permet de supporter plusieurs modules LoRa sur le même MCU : l'UART source est comparée au handle stocké pour identifier quel module a reçu des données. L'appel final à HAL_UART_Receive_IT() réactive l'interruption pour le prochain octet.

Point critique : La gestion du débordement de buffer (rx_index >= 255) remet simplement l'index à 0. Dans une version de production, une meilleure stratégie consisterait à signaler l'erreur ou à ignorer les données en excès plutôt que d'écraser le début du buffer.

6.3 LORA SendRaw() et LORA SendCommand() : Couche d'envoi

```
// Envoi brut sans attente de reponse
LORA_Status_t LORA_SendRaw(LORA_Handle_t *hlora, const char *cmd)
{
    if (!hlora || !cmd || !hlora->huart)
    {
        return LORA_STATUS_INVALID_PARAM;
    }
    if (HAL_UART_Transmit(hlora->huart, (uint8_t*)cmd, strlen(cmd),
        LORA_DEFAULT_TIMEOUT) != HAL_OK)
    {
        return LORA_STATUS_ERROR;
    }
    return LORA_STATUS_OK;
}

// Envoi + effacement du buffer + attente ACK
static LORA_Status_t LORA_SendCommand(LORA_Handle_t *hlora, const char *cmd,
    const char *ack, uint32_t timeout)
{
    __disable_irq();
    memset(hlora->rx_buffer, 0, LORA_RX_BUFFER_SIZE);
    hlora->rx_index = 0;
    __enable_irq();
    if (LORA_SendRaw(hlora, cmd) != LORA_STATUS_OK)
    {
        return LORA_STATUS_ERROR;
    }
    return LORA_WaitForResponse(hlora, ack, timeout);
}
```

Ces deux fonctions constituent la couche d'émission du driver. `LORA_SendRaw()` transmet simplement la commande AT sur l'UART en mode bloquant (`HAL_UART_Transmit` attend la fin de la transmission). Elle est utilisée pour les commandes ne nécessitant pas d'ACK, comme AT+LOWPOWER ou AT+RESET.

`LORA_SendCommand()` est la fonction de plus haut niveau. Elle commence par vider le buffer de réception dans une section critique (`__disable_irq` / `__enable_irq`) pour éviter de confondre une réponse précédente avec l'ACK de la nouvelle commande. Elle enchaîne ensuite l'envoi et l'attente de la réponse attendue.

6.4 LORA WaitForResponse() : Attente de réponse

```

static LORA_Status_t LORA_WaitForResponse(LORA_Handle_t *hlora,
                                          const char *expected,
                                          uint32_t timeout)
{
    uint32_t start = HAL_GetTick();
    char local_copy[LORA_RX_BUFFER_SIZE];
    while ((HAL_GetTick() - start) < timeout)
    {
        __disable_irq();
        strncpy(local_copy, hlora->rx_buffer, LORA_RX_BUFFER_SIZE - 1);
        __enable_irq();
        local_copy[LORA_RX_BUFFER_SIZE - 1] = '\0';
        if (strstr(local_copy, expected) != NULL) return LORA_STATUS_OK;
        HAL_Delay(1);
    }
    return LORA_STATUS_TIMEOUT;
}

```

Cette fonction implémente une attente active avec timeout. Elle interroge périodiquement le buffer de réception (copié dans une variable locale pendant une section critique) pour y chercher la chaîne ACK attendue via strstr(). La copie locale avant recherche est essentielle : elle évite qu'une interruption UART modifie le buffer pendant que strstr() est en cours d'exécution, ce qui pourrait provoquer une lecture de données incohérentes.

Une durée HAL_Delay de 1ms a été mise compte tenu des temps de réponse du module (de l'ordre de quelques dizaines de ms). En cas de non-réponse dans le délai imparti (LORA_DEFAULT_TIMEOUT = 1000 ms), la fonction retourne LORA_STATUS_TIMEOUT.

6.5 LORA P2P StartRX() : Démarrage de la réception

```

LORA_Status_t LORA_P2P_StartRX(LORA_Handle_t *hlora)
{
    LORA_Status_t s1 = LORA_P2P_SetMode(hlora);    // AT+MODE=TEST
    LORA_Status_t s2 = LORA_P2P_Config(hlora);    // AT+TEST=RFCFG,...
    if ((s1 && s2) != LORA_STATUS_OK) return s1;
    LORA_Status_t status = LORA_SendCommand(hlora, CMD_RECV,
                                           CMD_RECV_ACK,
                                           LORA_DEFAULT_TIMEOUT);

    if (status == LORA_STATUS_OK)
    {
        hlora->state = LORA_STATE_RX;
    }
    return status;
}

```

Cette fonction englobe la séquence complète de mise en réception : passage en mode TEST, configuration radio, puis activation de la réception continue (RXLRPKT). Le module reste ensuite en écoute permanente et répondra avec +TEST: RX "..." chaque fois qu'une trame LoRa valide sera reçue. La mise à jour de hlora->state permet de connaître l'état courant du module.

Attention nous ne devons pas mettre un délais d'attente dans le while de la réception après la configuration RX dans le main car cela cause un problème d'écoute permanent du module(perte de trame).

6.6 LORA P2P SendString() et LORA P2P SendHex() : Émission

```
// Envoi d'une chaine ASCII
LORA_Status_t LORA_P2P_SendString(LORA_Handle_t *hlor, const char *data)
{
    if (!hlor || !data)
    {
        return LORA_STATUS_INVALID_PARAM;
    }
    if (strlen(data) > 200)
    {
        return LORA_STATUS_INVALID_PARAM;
    }
    int len = snprintf(hlor->tx_buffer, LORA_TX_BUFFER_SIZE,
                      "%s,\"%s\"\\r\\n", CMD_SEND, data);

    if (len <= 0 || len >= LORA_TX_BUFFER_SIZE)
    {
        return LORA_STATUS_ERROR;
    }
    LORA_Status_t status = LORA_SendCommand(hlor, hlor->tx_buffer,
                                           CMD_SEND_ACK,
                                           LORA_DEFAULT_TIMEOUT);

    if (status == LORA_STATUS_OK)
    {
        hlor->state = LORA_STATE_TX;
    }
    return status;
}
```

La fonction SendString formate la commande AT+TEST=TXLRSTR,"données" en construisant la chaîne dans tx_buffer via snprintf(). La limite à 200 caractères est une protection préventive : au-delà, la trame RF deviendrait trop longue pour le SF/BW configuré. La vérification du retour de snprintf (len <= 0 ou débordement) protège contre les erreurs de formatage.

La variante LORA_P2P_SendHex() encode d'abord le tableau d'octets en hexadécimal ASCII (chaque octet devient 2 caractères, ex: 0xAB → "AB") avant de transmettre via AT+TEST=TXLRSTR. Ce format est utilisé pour envoyer des données binaires structurées.

6.7 LORA P2P Available() et LORA P2P Read() : Réception et parsing (décodage)

```
// Verification de disponibilite d'une trame recue
bool LORA_P2P_Available(LORA_Handle_t *hlor)
{
    char local[LORA_RX_BUFFER_SIZE];
    __disable_irq();
    memcpy(local, hlor->rx_buffer, LORA_RX_BUFFER_SIZE);
    __enable_irq();
    local[LORA_RX_BUFFER_SIZE - 1] = '\\0';
    return (strstr(local, "+TEST: RX \\") != NULL);
}
```

```
// Lecture et parsing d'une trame reçue
LORA_Status_t LORA_P2P_Read(LORA_Handle_t *hlora, LORA_P2P_Packet_t *packet)
{
    // Extraction RSSI et SNR depuis la ligne +TEST: LEN:N, RSSI:X, SNR:Y
    char *len_line = strstr(local, "+TEST: LEN:");
    sscanf(len_line, "+TEST: LEN:%d, RSSI:%hd, SNR:%hhd",
           &packet->rssi, &packet->snr);

    // Extraction du payload hex entre guillemets
    char *rx_line = strstr(local, "+TEST: RX ");
    char *first_quote = strchr(rx_line, '"');
    char *second_quote = strchr(first_quote + 1, '"');

    // Conversion hex ASCII -> octets binaires
    for (int i = 0; i < hex_len / 2; i++)
    {
        char byte_str[3] = { hex[i*2], hex[i*2+1], 0 };
        packet->payload[i] = (uint8_t)strtol(byte_str, NULL, 16);
    }
    memset(hlora->rx_buffer, 0, LORA_RX_BUFFER_SIZE);
    return LORA_STATUS_OK;
}
```

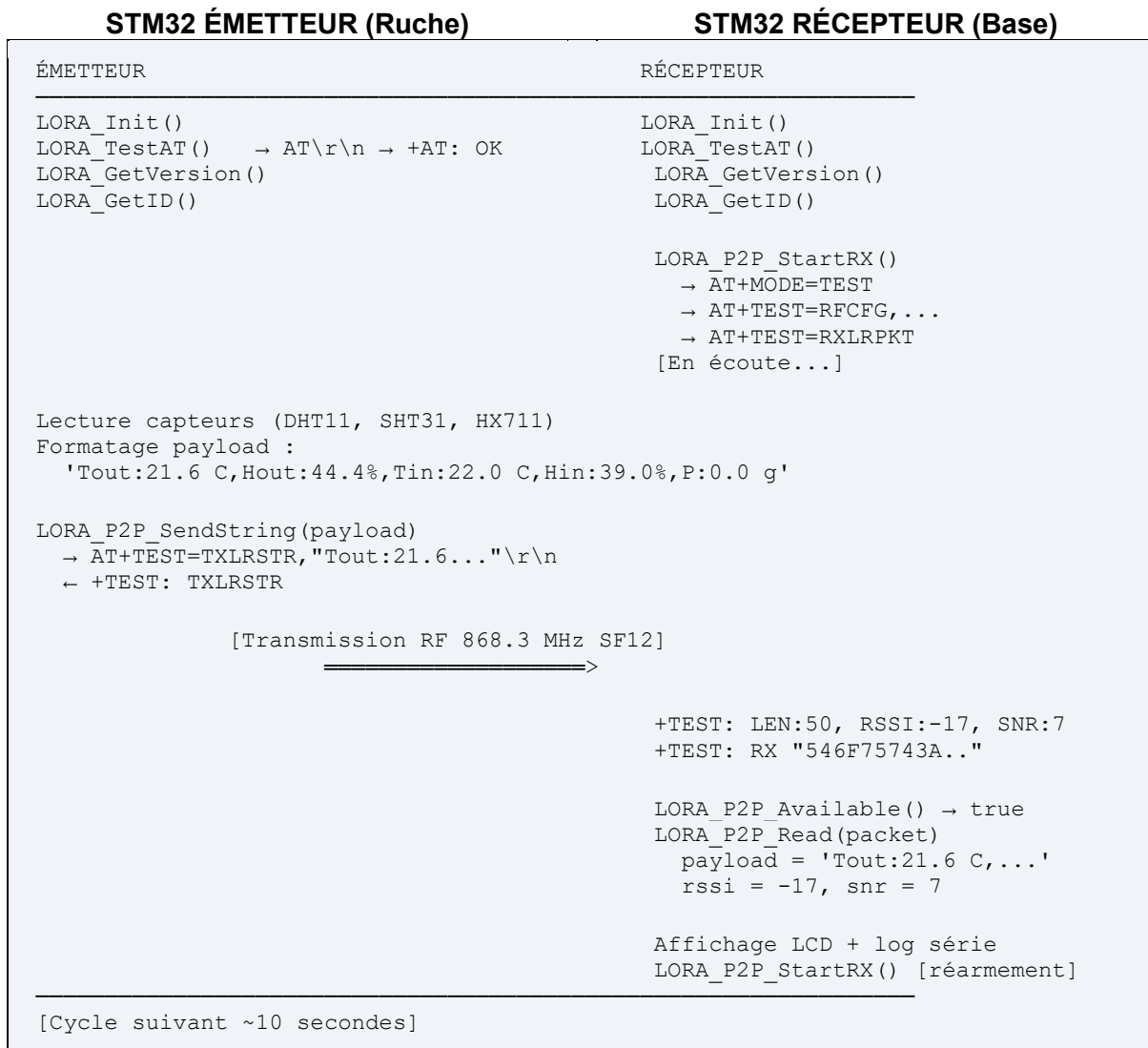
LORA_P2P_Available() est conçue pour être appelée dans la boucle principale sans bloquer : elle inspecte simplement le buffer via une copie locale protégée. La présence de la chaîne "+TEST: RX \"" indique qu'une trame complète a été reçue et est prête à être décodée.

LORA_P2P_Read() effectue un décodage en deux étapes. D'abord, elle extrait le RSSI et le SNR depuis la ligne +TEST: LEN en utilisant sscanf avec des formats d'entiers signés (%hd pour int16_t, %hhd pour int8_t). Ensuite, elle localise la chaîne hexadécimale entre guillemets dans la ligne +TEST: RX et la convertit octet par octet en binaire via strtol. Le buffer est effacé à la fin pour signaler que la trame a été consommée.

Format de réponse du module lors d'une réception : +TEST: LEN:50, RSSI:-17, SNR:7\n+TEST: RX "\"546F75743A32312E3620432C...\\""\n. Le payload est l'encodage hexadécimal ASCII de la chaîne de données de la ruche.

7. Séquence complète d'une transaction P2P

Le diagramme ci-dessous illustre la séquence complète d'échange entre les deux STM32 (émetteur ruche et récepteur base) et leurs modules LoRa-E5 respectifs, depuis l'initialisation jusqu'à la réception et l'affichage des données.



8. Intégration dans le projet ruche connectée

8.1 Architecture du système

La ruche connectée intègre trois capteurs complémentaires qui renseignent différents aspects de l'état de la colonie. Les données de ces trois capteurs sont agrégées par le STM32 émetteur, formatées en une chaîne ASCII et transmises via LoRa P2P.

Capteur	Variable transmise	Position	Rôle dans la ruche
DHT11	Tout, Hout	Extérieur ruche	Conditions ambiantes
SHT31	Tin, Hin	Intérieur ruche	MicroClimat de la colonie
HX711 (jauge)	P (poids)	Sous la ruche	Masse production de miel

8.2 Format du payload transmis

Le payload LoRa est une chaîne ASCII formatée par le MCU émetteur. Le format observé en fonctionnement est le suivant :

LoRa TX	[Tout:21.6 C , Hout:44.4 % , Tin:22.0 C , Hin:39.0 % , P:-0.0 g]
Tout	= Température extérieure (DHT11) en °C
Hout	= Humidité extérieure (DHT11) en %HR
Tin	= Température intérieure (SHT31) en °C
Hin	= Humidité intérieure (SHT31) en %HR
P	= Poids de la ruche (HX711) en grammes

Ce format ASCII lisible humainement facilite le débogage et l'affichage direct sur le LCD côté réception, sans besoin de décodage supplémentaire. La longueur typique du payload est d'environ 55 caractères.

```

=== LoRa P2P Started ===
Le Module LoRa est bien initialisé
Version du module : +VER: 4.0.11

ID du module :
+ID: DevAddr, 72:00:12:C6
+ID: DevEui, 2C:F7:F1:20:72:00:12:C6
+ID: AppEui, 52:69:73:69:6E:67:48:46

Le Module LoRa TestAT réussi
Le choix du Mode : P2P
Configuration P2P terminée
===== Lecture DHT11, SHT31, HX711 =====

+++++
+
+   LoRa TX   +
+
+++++

LoRa TX [Tout:21.6 C , Hout:44.4 % , Tin:22.0 C , Hin:39.0 % , P:-0.0 g ]
Paquet envoyé
===== Lecture DHT11, SHT31, HX711 =====

+++++
+
+   LoRa TX   +
+
+++++

LoRa TX [Tout:21.6 C , Hout:44.4 % , Tin:22.0 C , Hin:39.0 % , P: 0.0 g ]
Paquet envoyé

```

Figure 3 — Console série de l'émetteur : initialisation du module et cycles d'envoi des données ruche

8.3 Flux de données côté réception

Le STM32 récepteur exécute la séquence suivante à chaque cycle : vérification de disponibilité d'une trame (LORA_P2P_Available), lecture et parsing (LORA_P2P_Read), extraction des valeurs RSSI et SNR, affichage sur le terminal série et sur l'écran LCD local, puis réactivation de la réception.

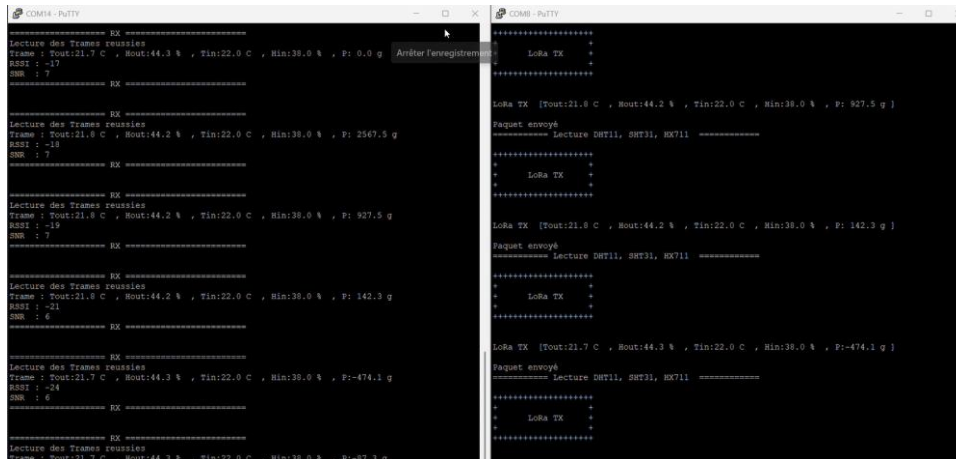


Figure 4 — Double terminal PuTTY : émetteur (droite) et récepteur (gauche) en fonctionnement simultané

La capture ci-dessus démontre la synchronisation parfaite entre les deux STM32 : chaque trame envoyée côté émetteur (droite) est immédiatement reçue et décodée côté récepteur (gauche), avec affichage du RSSI et du SNR associés.

9. Résultats expérimentaux

9.1 Analyse des mesures RSSI et SNR

Trame	RSSI (dBm)	SNR (dB)	Payload reçu
Trame 1	-17	7	Tout:21.7 C, Hout:44.3%, Tin:22.0, Hin:38.0, P:0.0 g
Trame 2	-18	7	Tout:21.8 C, Hout:44.2%, Tin:22.0, Hin:38.0, P:2567.5 g
Trame 3	-19	7	Tout:21.8 C, Hout:44.2%, Tin:22.0, Hin:38.0, P:927.5 g
Trame 4	-21	6	Tout:21.8 C, Hout:44.2%, Tin:22.0, Hin:38.0, P:142.3 g
Trame 5	-24	6	Tout:21.7 C, Hout:44.3%, Tin:22.0, Hin:38.0, P:-474.1 g

Les mesures ont été effectuées à une distance de 10 cm entre les deux modules, en environnement intérieur. Le RSSI varie entre -17 et -24 dBm, ce qui représente un signal très fort : à titre de référence, la sensibilité maximale du récepteur en SF12 est de -136 dBm. La marge de liaison disponible dépasse donc 110 dB, ce qui correspond théoriquement à une portée de plusieurs kilomètres en espace ouvert avec cette configuration.

Le SNR de 6 à 7 dB confirme d'excellentes conditions de réception. Toutes les trames ont été correctement reçues et décodées sans erreur de checksum côté LoRa-E5.

Note sur les valeurs de poids : Les variations importantes du poids (0.0 g, 2567.5 g, -474.1 g) observées dans les captures sont dues à l'absence de tare réalisée sur le capteur HX711 lors de ces tests. Le capteur de poids fonctionnel est calibré séparément.

9.2 Captures PicoScope : Analyse de la trame UART

Le PicoScope a été connecté sur la ligne UART entre le STM32 émetteur et le module LoRa-E5, afin d'observer physiquement les échanges de commandes AT.

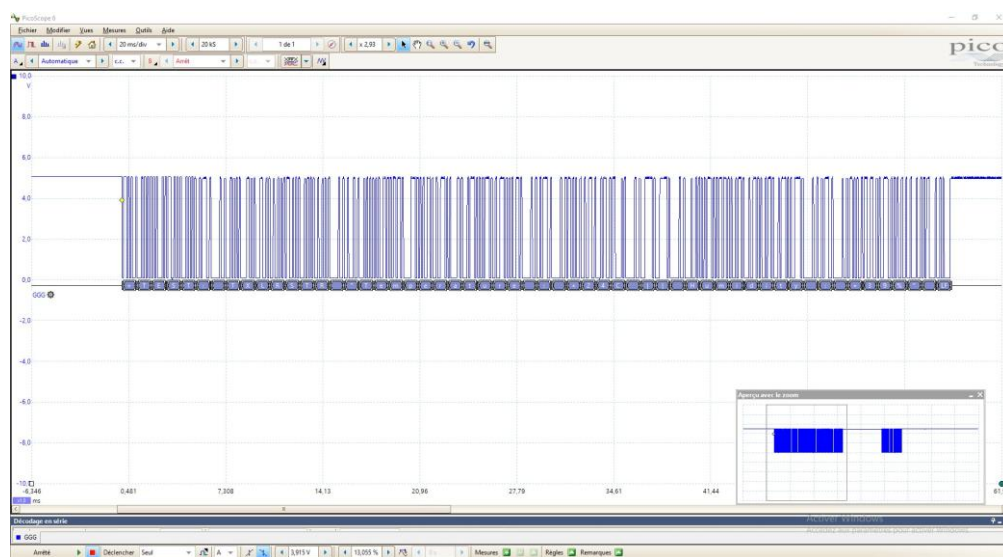


Figure 5 — PicoScope : trame UART complète de la commande AT+TEST=TXLRSTR envoyée au module LoRa-E5

La Figure 5 montre la séquence UART complète lors d'un envoi : le signal oscillant entre 0 et ~5V correspond aux bits UART série à 9600 baud. Le décodage série visible en bas de l'écran révèle les caractères ASCII de la commande AT. La durée totale de la séquence (visible sur l'axe temporel gradué en ms) est cohérente avec la longueur du payload et la vitesse de transmission.

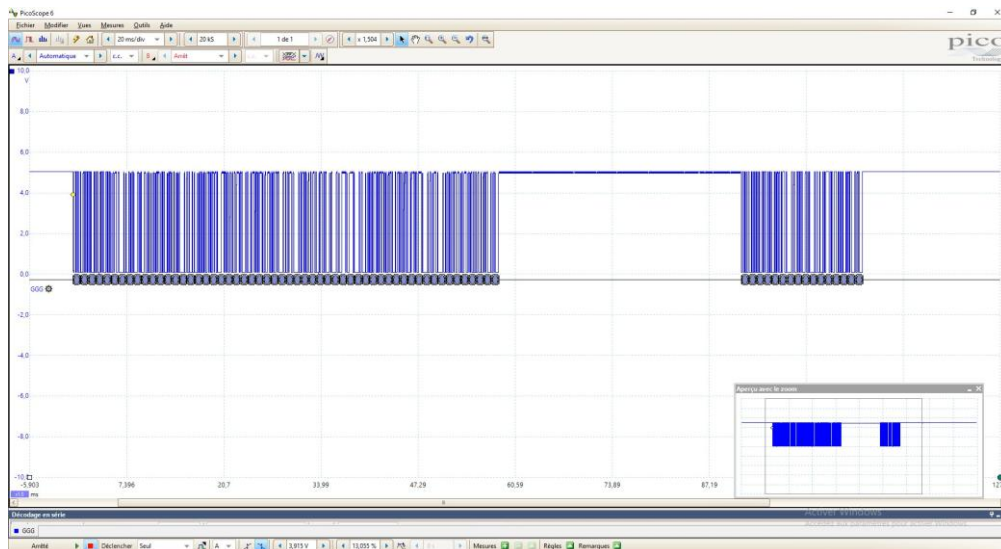


Figure 6 — PicoScope : commande AT (burst gauche), silence RF, puis ACK du module (burst droit)

La Figure 6 est particulièrement révélatrice de la mécanique interne du driver. On distingue clairement trois phases : le premier burst à gauche correspond à la commande AT+TEST=TXLRSTR envoyée par le STM32 au module (fonction LORA_SendRaw). La zone de silence d'environ 30 ms au centre est le temps de traitement radio du module LoRa-E5 ; c'est pendant ce silence que la trame LoRa est effectivement modulée et émise à 868,3 MHz. Le burst à droite est la réponse ACK +TEST: TXLRSTR renvoyée par le module au STM32, capturée par LORA_WaitForResponse().

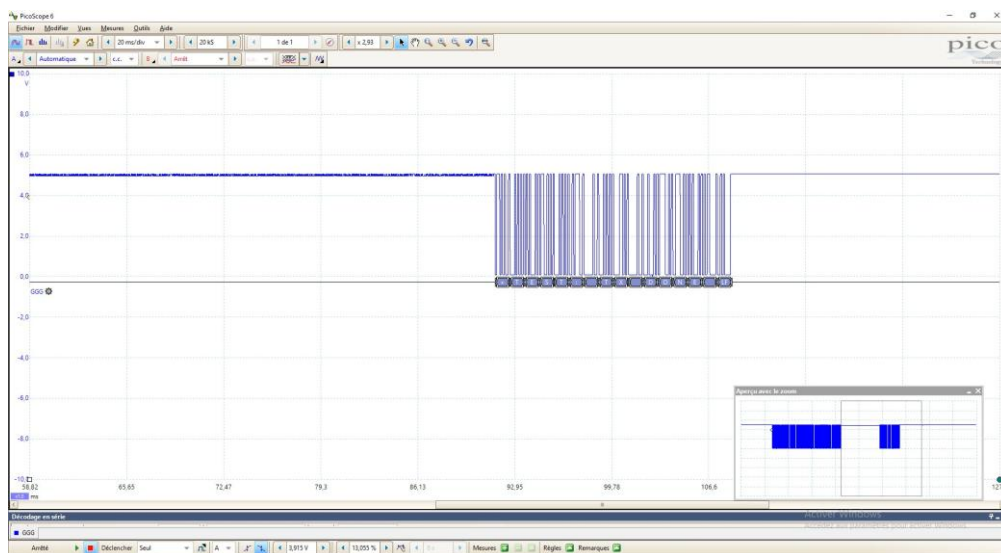


Figure 7 — PicoScope : zoom sur l'ACK du module — décodage des caractères ASCII +TEST=TXLR visible

La Figure 7 présente un zoom sur le burst de réponse ACK. Le décodage ASCII en bas de l'écran permet d'identifier les caractères "+TEST=TX DONE" de la réponse pour signaler que la trame a été bien envoyée. On constate la brièveté de cette réponse (quelques dizaines de ms) comparée à la commande initiale plus longue.

10. Perspectives — Mode LoRaWAN

10.1 Principes du mode LoRaWAN

En complément du mode P2P, le module LoRa-E5 supporte nativement le protocole LoRaWAN (Classes A, B et C). Contrairement au mode P2P, LoRaWAN requiert une infrastructure réseau : une gateway LoRaWAN (passerelle RF/IP) et un serveur réseau (LNS — LoRaWAN Network Server, comme The Things Network ou ChirpStack). Ce mode apporte la sécurité AES-128, la gestion des compteurs de trames, et l'interopérabilité avec les réseaux IoT publics.

10.2 Modes d'activation

Mode	Description	Commandes AT concernées
LWOTAA	Over-The-Air Activation — le device négocie ses clés via le réseau à chaque join	AT+MODE=LWOTAA, AT+JOIN
LWABP	Activation By Personalization — clés pré-configurées en dur (NwkSKey, AppSKey)	AT+MODE=LWABP, AT+KEY=...

10.3 Statut dans ce projet

Information importante : Le driver LoRaWAN (LoRa_E5_WAN.c / .h) a été rédigé et intégré au dépôt du projet. Cependant, il n'a pas pu être testé lors de ce bureau d'étude. La validation de ce module est prévue dans une phase ultérieure du projet, lors de la mise en service en conditions réelles avec une gateway TTN (The Things Network).

Les commandes AT nécessaires au mode LoRaWAN sont toutes supportées par le firmware 4.0.11 utilisé dans ce projet. La migration du mode P2P vers LoRaWAN ne nécessitera que la modification du mode d'initialisation et la configuration des clés de sécurité, le reste du driver restant identique.